

The F2Z forest GKR: an algorithmic description

The `f2z-pcs` repository

2026-08-15

Abstract

This is the *algorithmic* companion to the prover-engineering note `docs/forest-gkr-note`: the protocol the merged grand-product forest performs, written in the order the prover and verifier execute it — the statement, the entry, the layer loop, the exit discharge into the ring-switch/Ligerito opening, the read-off, and the soundness chain — followed by a self-contained account of the lookup-table machinery by which the prover runs the bottom of the forest without ever materialising it. Kernel internals, software prefetch and the full measured cost tables live in the companion note; nothing in the execution machinery described here changes a transcript byte. Everything here is implemented in `src/merged_forest.rs`, `src/piop/sumcheck/eq_factored.rs` and `src/ligerito.rs`; §4 maps the text to function names.

Contents

1	What the forest certifies	2
2	The protocol	2
2.1	Entry: integers, roots, one claim	2
2.2	The descent: one sumcheck per level	3
2.3	The exit claim and its discharge	4
2.4	Read-off and the soundness chain	4
2.5	Chunking: why there can be several forests	5
3	Execution: the lookup-table prover	5
3.1	What a round and a fold are	5
3.2	Low entropy and the width laws	7
3.3	What the prover holds	8
3.4	Use 1: round messages as gathers (the leaf round, worked)	9
3.5	Use 2: folds kept as tables (the stash)	11
3.6	Use 3: generating the dense region from T_4	13
3.7	Why the transcript is unchanged	14
4	Where this sits in the code	15

1 What the forest certifies

F2Z proves an integer multilinear-extension evaluation $\widetilde{\text{INT}}(D)(r) = y \in \mathbb{F}_q$ over data D whose *bits* are committed with a characteristic-2 commitment. The instance shape is (t, s, W) : D is a $2^t \times 2^s$ matrix of W -bit cells, arranged as the single-bit matrix

$$M[c][i] \in \{0, 1\}, \quad i = (b \ll \log_2 W) \mid j \in \{0, 1\}^d, \quad d = t + \log_2 W,$$

with b the row, c the column, j the bit position inside the cell. Splitting the evaluation point r into its row and column halves, the claim decomposes as

$$y = \sum_c \text{eq}(c, r_{\text{col}}) \cdot u_c \bmod q, \quad u_c = \sum_b w_b \cdot \text{INT}(D(b, c)), \quad w_b = \text{eq}(b, r_{\text{row}}) \bmod q$$

with the row weights w_b lifted to integers in $[0, q)$. The prover will simply *send* the per-column integers and let the verifier do the column combination in the clear; the entire difficulty is certifying the sent integers against the $\mathbb{F}_2$ commitment, since parities destroy magnitudes. F2Z carries the integer fold in the exponent of $\mathbb{K} = \text{GF}(2^{128})$, with $\alpha \in \mathbb{K}^\times$ a fixed public generator.

For one chunk of the mod- q opening (chunking is §2.5; read this section at $L = 1$, i.e. $W_b = w_b$), with public c_w -bit weights W_b , define the public *leaf constants* and the *leaves*

$$\tau(i) = \alpha^{W_b 2^j} - 1, \quad \Lambda[c][i] = 1 + M[c][i] \cdot \tau(i) \in \{1, \alpha^{W_b 2^j}\} \quad (1)$$

(in $\mathbb{K}$, $-1 = +1$; the $-$ form is kept for readability). Then, purely by construction,

$$\prod_{i \in \{0,1\}^d} \Lambda[c][i] = \alpha^{\sum_{b,j} W_b 2^j M[c][i]} = \alpha^{u_c}. \quad (2)$$

So “the sent integers are the true weighted folds of the committed bits” *is* the statement that α^{u_c} equals a grand product of 2^d factors, each of which is 1 or a public constant *selected by one committed bit* — for all 2^s columns at once. That is the claim the forest certifies. The factors being *bit-affine* in the committed data (degree 1 in each bit) is what every later step leans on.

2 The protocol

The proof is a product-tree GKR in the lineage of [1, 2]: one binary product tree per column, walked from the roots *down* to the leaves, one sumcheck per tree level. Three F2Z-specific choices shape it: (i) the roots are never sent — the verifier recomputes them as α^{u_c} from the sent integers; (ii) the 2^s trees are *merged*: the column index is just s more multilinear variables, so one sumcheck per level covers every tree and nothing per-tree ever enters the transcript; (iii) the leaf layer is never committed — because leaves are bit-affine, the final leaf claim converts into a bare evaluation of the committed bit matrix, discharged by the PCS opener.

2.1 Entry: integers, roots, one claim

1. $P \rightarrow V$: the integer folds u_c for every column c (16 bytes each; the **us** vector of the proof stream).
2. V : range-check every $u_c < 2^{c_w+t+W} = 2^{127}$, and check that α generates $\mathbb{K}^\times$. Together these make $x \mapsto \alpha^x$ *injective* on every value that can occur ($\text{ord } \alpha = 2^{128} - 1$).

3. Both sides: $R_c \leftarrow \alpha^{u_c}$ (the verifier by a fixed-base comb from the sent integers; the honest prover has them as the tops of its trees — their equality is precisely the claim under proof), then

$$\mathbf{absorb}(0x30 : R_0, \dots, R_{2^s-1}), \quad \zeta \leftarrow \mathbf{squeeze}(\mathbb{K}^s), \quad C \leftarrow \widetilde{R}(\zeta),$$

each side computing the entry claim $C = \widetilde{R}(\zeta)$ itself ($O(2^s)$ work). Set $z_x \leftarrow ()$, $z_c \leftarrow \zeta$.

The absorb is the binding: a prover whose later messages were generated against different roots faces challenges derived from *these* roots. And instead of 2^s separate root claims, the verifier holds from here on exactly *one* running claim, about the MLE of the root vector at a random point.

2.2 The descent: one sumcheck per level

Index the values of tree c at depth ℓ (root = 0, leaves = d) by $x \in \{0, 1\}^\ell$, giving one multilinear family per level, $L_\ell(x, c)$ with x the low variables and c the high ones, related by the product recursion (children paired on the *top* in-tree bit: node x 's children sit at positions x and $x + 2^\ell$ of level $\ell + 1$):

$$L_\ell(x, c) = \underbrace{L_{\ell+1}(x, 0, c)}_{E_c(x)} \cdot \underbrace{L_{\ell+1}(x, 1, c)}_{O_c(x)}. \quad (3)$$

$L_0(\cdot, c) = R_c$ are the roots and $L_d = \Lambda$ the leaves.

Invariant. Entering layer ℓ the verifier holds one claim $C = \widetilde{L}_\ell(z_x, z_c)$, $z_x \in \mathbb{K}^\ell$, $z_c \in \mathbb{K}^s$. By (3) and multilinearity it factors coordinate-wise:

$$C = \sum_{x \in \{0,1\}^\ell} \sum_{c \in \{0,1\}^s} \mathbf{eq}(x, z_x) \mathbf{eq}(c, z_c) E_c(x) O_c(x). \quad (4)$$

The layer converts this into the same kind of claim one level down:

1. **Phase A** (the ℓ in-tree variables; skipped at $\ell = 0$). A sumcheck over x of (4), binding $x \rightarrow r_x \in \mathbb{K}^\ell$; every round polynomial has degree 3 ($\mathbf{eq} \cdot E \cdot O$). Verifier checks: phase A's claimed sum equals the running claim C , and its final value equals $\mathbf{eq}(r_x, z_x) \cdot (\text{phase B's claimed sum})$.
2. **Phase B** (the s tree variables). A sumcheck over c of the residual $\sum_c \mathbf{eq}(c, z_c) \widetilde{E}_c(r_x) \widetilde{O}_c(r_x)$, binding $c \rightarrow r_c \in \mathbb{K}^s$.
3. **Closing pair and line step.** $P \rightarrow V$: the two children

$$(p, q) = (\widetilde{L}_{\ell+1}(r_x, 0, r_c), \widetilde{L}_{\ell+1}(r_x, 1, r_c)).$$

Verifier checks phase B's final value = $\mathbf{eq}(r_c, z_c) \cdot p \cdot q$. Then both sides run

$$\mathbf{absorb}(0x32 : p, q), \quad \lambda \leftarrow \mathbf{squeeze}(\mathbb{K}), \\ C \leftarrow p + \lambda(q - p), \quad z_x \leftarrow r_x \parallel \lambda, \quad z_c \leftarrow r_c.$$

The line step is the standard two-to-one reduction: the sumcheck leaves *two* evaluations of level $\ell + 1$ (the children at $(r_x, \cdot, r_c)$); folding them along the paired variable with a fresh λ gives back a single claim $C = \widetilde{L}_{\ell+1}(r_x \parallel \lambda, r_c)$, restoring the invariant. (The code calls λ **mu**; $- = +$ in char 2.)

After d layers the loop exits with the *exit claim*

$$e_d = C \stackrel{\text{honest}}{=} \widetilde{\Lambda}(z, z_c), \quad z = (z_x, z_c) \in \mathbb{K}^{d+s},$$

one evaluation of the leaf MLE at a random point.

Remark 1 (Merging). *A naive realisation runs 2^s independent GKR and pays a per-tree child pair every layer — $O(2^s d)$ proof. Treating the tree index as s more MLE variables makes the per-layer payload $O(\ell + s)$ round messages plus one child pair, and phase A’s per-tree finals $(\widetilde{E}_c(r_x), \widetilde{O}_c(r_x))$ stay prover-internal — nothing per-tree is ever absorbed or sent. At the deployed shapes the sent folds ($16 \cdot 2^s$ bytes) are the larger forest-side proof item.*

Remark 2 (The A/B split is bookkeeping, not protocol). *Semantically each layer is one $(\ell + s)$ -variable sumcheck of (4); the implementation processes the x variables first (the 2^s trees run as eq-weighted groups sharing the point z_x , each tree’s $\text{eq}(c, z_c)$ riding as a per-group scalar) and then the c variables over the tiny per-tree finals. The split is what the lazy per-tree round bodies of §3 and the across-tree parallelism need; the transcript cost is one extra sumcheck header per layer.*

2.3 The exit claim and its discharge

The leaf layer never becomes an oracle. Substituting (1) into the exit claim, splitting $\text{eq}((i, c), z) = \text{eq}(i, z_x) \text{eq}(c, z_c)$ and using $\sum \text{eq} = 1$:

$$e_d - 1 = \sum_{i \in \{0,1\}^d} R[i] \cdot m[i], \quad \begin{aligned} R[i] &= \text{eq}(i, z_x) \tau(i) && \text{(public, size } 2^d), \\ m[i] &= \sum_c \text{eq}(c, z_c) M[c][i] && \text{(the } z_c\text{-combined rows).} \end{aligned} \quad (5)$$

The exit claim is thus already a $\mathbb{K}$ -linear functional of the committed bits — but against a *dense, unstructured* weight vector R (the α -powers destroy any tensor structure), which the opener cannot consume: it proves plain MLE *point* evaluations. One last **degree-2 sumcheck** (the “presum”) separates the factors:

1. Run a d -round degree-2 sumcheck on $\sum_i R(i) m(i) = e_d - 1$, binding $i \rightarrow r^*$. (Verifier checks its claimed sum against the forest’s e_d .)
2. V : evaluate $\widetilde{R}(r^*)$ *itself* — the verifier’s one linear-cost step, $O(2^t \cdot W)$, with the α^{W_b} bases from the same fixed-base comb; reject if $\widetilde{R}(r^*) = 0$. Set

$$\mu \leftarrow \frac{\text{final sumcheck value}}{\widetilde{R}(r^*)}.$$

3. The residual claim is $\widetilde{M}(r^* || z_c) = \mu$ — a *bare MLE evaluation of the committed bit matrix* at a *random point*, every α -dependent factor moved to the verifier’s side of the equation.

From here the opening is out of the forest’s hands: the ring switch sends 128 partial evaluations s_v (checked against μ), the L chunks’ residual claims are η -combined, and *one* recursive Ligerito call certifies the batch against the Merkle commitment (black box here; verifier note §6).

2.4 Read-off and the soundness chain

The verifier finishes in the clear, in $\mathbb{F}_q$:

$$y = \sum_c \text{eq}(c, r_{\text{col}}) \cdot \left(\sum_l 2^{c_w l} u_c^{(l)} \right) \bmod q,$$

checked against the claimed value. The chain, bottom-up — each link one of the steps above:

- *Ligerito opening* $\Rightarrow \mu$ is the true $\widetilde{M}(r^*||z_c)$ of the committed bits;
- *presum + the verifier's own $\widetilde{R}(r^*)$* $\Rightarrow$ the exit claim e_d is the true $\widetilde{\Lambda}(z)$;
- *d layer sumchecks* $\Rightarrow \widetilde{R}(\zeta)$ is the true MLE of the actual products $\prod_i \Lambda[c][i]$;
- *the absorb + random ζ* $\Rightarrow$ each root R_c equals its product (Schwartz–Zippel over $\mathbb{K}$);
- $R_c := \alpha^{u_c}$, *the range check, the generator check* $\Rightarrow$ the sent u_c are the integer folds of the committed bits (injectivity of the exponent map below $\text{ord } \alpha$);
- *recombination* $\Rightarrow y = \widetilde{\text{INT}}(D)(r) \bmod q$.

Both the range check and the generator check are load-bearing: without either, $\alpha^u = \alpha^{u'}$ stops implying $u = u'$.

Costs at a glance. Verifier: $O(2^s)$ exponentiations for the roots + $O(2^s)$ for the entry MLE and the read-off, $O(\sum_\ell(\ell + s)) = O(d^2/2 + ds)$ round checks, and the one $O(2^t W)$ evaluation of $\widetilde{R}(r^*)$ — milliseconds at every deployed shape. Proof, forest side: the folds $16 \cdot 2^s$ B plus $\approx 32(\frac{d(d-1)}{2} + ds)$ B of round messages and $32d$ B of pairs, per chunk. Prover: the forest + presum phase is $O(2^{d+s})$ field work and 81–96 % of an F2Z prove; §3 is about how it is actually spent.

2.5 Chunking: why there can be several forests

A full $\sim q_{\text{bits}}$ -bit weight w_b would let $\sum_b w_b \text{INT}(D(b, c))$ overflow 2^{127} and break injectivity. So the weights are split into base- 2^{c_w} limbs, $c_w = 127 - t - W$, $w_b = \sum_l 2^{c_w l} W_b^{(l)}$, and *everything in §§2.1–2.3 runs once per limb* on the same transcript and the same commitment: per-limb integers $u_c^{(l)}$, per-limb forest, per-limb presum, each chunk-fold satisfying $u_c^{(l)} < 2^{c_w + t + W} = 2^{127}$ by construction. The L residual bit-MLE claims (at different points) are η -RLC'd into the single Ligerito call, and the verifier reassembles $\sum_l 2^{c_w l} u_c^{(l)}$ in the read-off. At the deployed shapes ($q = 2^{100} - 15$, $t \leq 26$) one has $c_w \geq 100$, so $L = 1$: the whole scheme can be read with one chunk in mind.

3 Execution: the lookup-table prover

Nothing below changes the protocol. The tables exist to execute *phase A of the bottom three layers* (and the level build beneath them) — the rounds whose operand buffers E_c, O_c would otherwise be the two bottom levels of the forest, 2^{d+s} and 2^{d+s-1} $\mathbb{K}$ -elements (4 GiB + 2 GiB at $n = 28$). Their messages and folds are computed *as if* those buffers existed, from lookups keyed by committed bits; the transcript is byte-identical either way (pinned by `lazy_matches_eager` in every schedule).

3.1 What a round and a fold are

Fix one layer ℓ and one tree, and look at its phase A through the prover's eyes. It is an ordinary k -variable sumcheck, $k = \ell$, of

$$S = \sum_{x \in \{0,1\}^k} w(x) E(x) O(x), \quad w(x) = \text{eq}(x, z_x),$$

with E, O — the tree's two halves of the input level — held as two arrays of 2^k field elements. The prover never manipulates a polynomial symbolically; everything it does is one of two passes over these arrays.

A round is one message–challenge exchange. The sumcheck runs k rounds, one per variable, lowest index first (a *round* is a step *within* one layer’s phase A, not a layer). In round j the prover sends the univariate

$$P_j(X) = \sum_{y \in \{0,1\}^{k-j}} (\tilde{w} \tilde{E} \tilde{O})(\rho_1, \dots, \rho_{j-1}, X, y)$$

— the variables already bound are fixed at the earlier challenges, the current one is left symbolic, the tail is still summed over the cube; degree 3 here, each factor being multilinear. The verifier checks $P_j(0) + P_j(1)$ against its running claim, draws ρ_j , and the claim becomes $P_j(\rho_j)$. After k rounds nothing is left to sum: the claim concerns the single random point $r_x = (\rho_1, \dots, \rho_k)$.

The prover’s state, and slots. P_j is a sum, over the remaining cube, of products of *restrictions* of multilinear — and a multilinear restricted to $(\rho_1, \dots, \rho_{j-1}, X, b)$ is *affine in X* , determined by its two endpoints at $X = 0, 1$. The prover therefore keeps each array in the state “bound prefix substituted, suffix still Boolean”,

$$E_j[y] = \tilde{E}(\rho_1, \dots, \rho_{j-1}, y), \quad y \in \{0,1\}^{k-j+1}$$

(E_1 is the raw input), because those endpoints are then *literally stored entries*: grouping the entries into *slots* — slot b is the adjacent pair $(2b, 2b+1)$, the two settings of the current variable —

$$\tilde{E}(\rho_1, \dots, \rho_{j-1}, X, b) = E_j[2b] + X (E_j[2b+1] - E_j[2b]),$$

and likewise for O and w — though the w -line, unlike the other two, will need no stored array at all (the closing paragraph of this section) — whence

$$P_j(X) = \sum_{b \in \{0,1\}^{k-j}} [w\text{-line}](X) \cdot [E\text{-line}](X) \cdot [O\text{-line}](X).$$

One sweep over the slots — two stored entries per slot per array — accumulates the message’s coefficients: the A_0, A_1, A_2 of §3.4, the w factor riding as the precomputed suffix weights ω_b . (At Boolean points the entries *are* the current summand values, which is why $P_j(0) + P_j(1)$ matches the running claim for the honest prover.) Keeping the arrays in this state is what makes the prover *linear-time*: round j ’s passes cost $O(2^{k-j+1})$, the sizes halve, so a layer totals $O(2^k)$ — whereas recomputing the restrictions from the raw arrays each round (each endpoint a fresh 2^{j-1} -term eq-weighted sum) would pay an extra factor of k .

A fold is the array update after the challenge. Once ρ_j arrives, substitute it — evaluate every slot’s line at $X = \rho_j$:

$$E_{j+1}[b] = E_j[2b] + \rho_j (E_j[2b+1] - E_j[2b]), \tag{6}$$

one linear pass that merges each pair into one entry and *halves* the array, $2^{k-j+1} \rightarrow 2^{k-j}$. Rounds are the protocol’s clock; folds are the bookkeeping tick that advances the arrays to the next round’s state. A k -variable phase A thus walks $2^k \rightarrow 2^{k-1} \rightarrow \dots \rightarrow 1$, and the last fold’s single entries are the per-tree finals $\tilde{E}(r_x), \tilde{O}(r_x)$ handed to phase B. Unrolling (6): after f folds,

$$E_{f+1}[b] = \sum_{\varepsilon \in \{0,1\}^f} \text{eq}((\rho_1, \dots, \rho_f), \varepsilon) \cdot E_1[2^f b + \varepsilon],$$

a *fixed* $\mathbb{K}$ -linear combination — coefficients global across trees and positions — of one *contiguous, aligned block* of 2^f round-1 entries. (The alignment is why the LUT rounds below read their case indices as aligned 2-bit, nibble and byte windows of the committed bits.)

The w -line needs no table. The eq weight is the one factor whose line is never read from a stored array, because eq factors coordinate-wise: $\text{eq}(x, z) = \prod_i \text{eq}_1(x_i; z_i)$ with $\text{eq}_1(x; z) = zx + (1-z)(1-x)$, degree 1 in each variable separately. Writing $z_1, \dots, z_k$ for the coordinates of z_x , the restriction to round j , slot b splits along exactly those lines:

$$\tilde{w}(\rho_1, \dots, \rho_{j-1}, X, b) = \underbrace{\prod_{i < j} \text{eq}_1(\rho_i; z_i)}_{A_j} \cdot \underbrace{\text{eq}_1(X; z_j)}_{\text{public, slot-independent}} \cdot \underbrace{\prod_{i > j} \text{eq}_1(b_{i-j}; z_i)}_{\omega_b = V_j[b]}$$

and each piece has a different fate. The *prefix scalar* A_j depends only on past challenges: one running field element, one multiply per round ($A_{j+1} = A_j \text{eq}_1(\rho_j; z_j)$) — and in the multi-tree phase A it is seeded with the tree’s group scale $\text{eq}(c, z_c)$, which is how the merged layer’s per-tree weights ride along for free. The *current-variable factor* $\text{eq}_1(X; z_j)$ carries all of the w -line’s X -dependence and is the same public line for every slot, so it is pulled outside the sum — the prover never multiplies by it at all (next paragraph). The *suffix weight* $\omega_b = V_j[b]$ is the only per-slot part; since b is Boolean, each factor is z_i or $1 - z_i$, so V_j is a tensor, and all k of them are precomputed back-to-front ($V_k = [1]$, $V_j = \text{eq}_1(\cdot; z_{j+1}) \otimes V_{j+1}$; `suffix.tensors`) in $O(2^k)$ total — the cost of one eq-table build — once per phase-A call and shared by every tree, since all trees share the point z_x . The round polynomial therefore factors,

$$P_j(X) = \text{eq}_1(X; z_j) \cdot \hat{H}_j(X), \quad \hat{H}_j = \sum_c A_{c,j} \sum_b \omega_b [E\text{-line}] [O\text{-line}],$$

with $\hat{H}_j$ quadratic: the (A_0, A_1, A_2) of §3.4 are its per-tree coefficients, and the entire per-slot w handling is one scalar weighting folded into the E side — the 5-multiply schedule $\omega e_0, \omega e_1, (\omega e_0) o_0, (\omega e_1) o_1, \Delta E \Delta O$. “Folding w ” costs nothing pass-shaped: one multiply into the prefix scalar and a step to the next, half-length suffix tensor.

Where the public factor ends up: the verifier’s side. Because $\text{eq}_1(X; z_j)$ is public, the prover sends only $\hat{H}_j$ ’s two non-constant coefficients $(\hat{H}_1, \hat{H}_2)$ — Gruen’s factored round format. The verifier recovers the constant from its running claim: expanding $S_j = P_j(0) + P_j(1) = (1-z_j)\hat{H}(0) + z_j\hat{H}(1)$, the $\pm z_j\hat{H}_0$ terms cancel, leaving

$$\hat{H}_0 = S_j - z_j(\hat{H}_1 + \hat{H}_2)$$

— an identity over any field, no inversion — after which it advances $S_{j+1} = \text{eq}_1(\rho_j; z_j) \cdot \hat{H}_j(\rho_j)$ (`verify_eq_inner_sumcheck_gruen`). Soundness is unchanged: the prover is thereby restricted to degree-3 polynomials divisible by the public linear factor — a *subset* of the generic message space, so the standard round analysis applies verbatim.

3.2 Low entropy and the width laws

Everything rests on one observation: a node r levels above the leaves is the product of the 2^r leaves beneath it, and each leaf is 1 or the public constant $\alpha^{W_b 2^j}$ — so the node’s value is one of 2^{2^r} *position-indexed constants, selected by 2^r committed bits, independent of the tree*. One shared table can serve all 2^8 trees; per tree one only reads bits.

The second ingredient is what a fold (§3.1) does to such an entry: after f folds, an array entry is a fixed $\mathbb{K}$ -linear combination of 2^f round-1 entries with globally shared coefficients — so if each

round-1 entry was selected by w committed bits out of a shared position table, the folded entry is selected by the $2^f w$ bits of its block, from a new position table with the challenges baked in. Folding a bit-affine leaf pair makes this concrete — using $(1+\rho) + \rho = 1$ in char 2:

$$v' = (1+\rho)(1 + m_0\tau_0) + \rho(1 + m_1\tau_1) = 1 + m_0(1+\rho)\tau_0 + m_1\rho\tau_1, \quad (7)$$

a 4-case function of the bit pair (m_0, m_1) , again from a position-indexed, tree-shared table (now with ρ baked in). Hence two *width laws*, each of which squares the case count: going up a level ($2^w \rightarrow 2^{2w}$ cases, the parent coupling two selects) and folding a round (identically). An entry of level $d - r$ after f folds depends on 2^{r+f} committed bits — r *fixed* for a layer (how far above the leaves its input level sits), f *advancing* $0, 1, 2, \dots$ with the layer’s own rounds — and its case table holds $2^{2^{r+f}}$ cases per position:

$r + f$	bits/entry	cases	table at $d = 17$	
0	1	2	2 MiB	the τ halves
1	2	4	4 MiB	te, to
2	4	16	8 MiB	T4
3	8	256	≈ 128 MiB	— not built
4	16	65 536	≈ 8 GiB	— exceeds what it would replace

Why the cap is $r + f \leq 2$. The cap is *not* a soundness or correctness constraint — any depth would emit the identical transcript. It is the economics knee of a doubly-exponential blow-up (+1 *squares* the case count), set by three compounding facts. (i) *Cache residency*. The value of a shared table is that all 2^s trees gather from it with data-dependent (bit-selected) indices; that pays only while the table is cache-resident. The measured rule of thumb is that one gather sweep over the 8 MiB *T4* already costs about one *streaming* pass over the level it replaces; a 128 MiB table turns every gather into a DRAM miss and costs *more* than the dense computation — the rounds are gather-bound, not multiply-bound. (Direct evidence that the knee is residency: even within the cap, the 16-case precombined leaf $\Delta\Delta$ table stops paying once it outgrows L2 and the code auto-switches to a factored 4-entry form, §3.4.) (ii) *Build amortisation*. A table costs one multiply per entry to build; at $r+f = 3$ the build makes $32\times$ more values than the level it serves has nodes per tree, eroding the 2^s -fold amortisation before the miss costs are even counted. (iii) *Diminishing returns*. Each +1 buys one more round run off bits — one more halving of where dense buffers are born. The 16-case machinery already runs three rounds off the leaves; the marginal fourth would halve an already-small object at the price of the 128 MiB table. Where a round *message* couples several entries into a wider select (the pair round’s $\Delta E \cdot \Delta O$ term couples 8 bits), the code does not build the 256-case table either: it *factors* the term into two 16-case tables joined by one explicit multiply. Sixteen cases is the last table size that earns its bytes; anything wider is recovered by arithmetic on 16-case pieces. (A separate, structural guard — a prefix of f LUT rounds needs $k \geq f+1$ variables — just says the last LUT round must still have a fold in which the dense buffers get born; it governs shallow trees, not the cap.)

3.3 What the prover holds

All shared tables are megabytes, amortised over the 2^s trees; the per-tree state during the LUT rounds is *only the bit arrays*. Write $q_1 = 2^{d-2}$, $q_2 = 2^{d-1}$.

object	contents	index	at $d=17$
lbits/rbits (per tree)	the column's bits, split at the midpoint	bit i , 64/word	32 MiB total
v, τ halves	$v(i) = \alpha^{W_b 2^j}$, $\tau = v - 1$; case 0 ($= 1$) unstored	position i	2 MiB
pair	$v(i) v(i+q_2)$ — the only level- $(d-1)$ case not in v	$i < q_2$	1 MiB
te, to	the 4 cases $\{1, \tau_{lo}, \tau_{hi}, \tau_{lo}\tau_{hi}\}$ of a level- $(d-1)$ node (E/O halves; O's offset baked in)	$4y \mid (m_{lo} \mid m_{hi} \ll 1)$	4 MiB
$T4$	all 16 values of a level- $(d-2)$ node (te · to)	$(y \ll 4) \mid (c_E \ll 2) \mid c_O$	8 MiB
per-round tables	suffix-eq weights ω_b and past challenges baked into case tables (§3.4–3.5)	slot $\times$ case	MiB-scale

The midpoint split of the bits is the top-bit pairing convention of (3) made physical: a level- $(d-1)$ node at position i multiplies leaves i and $i + q_2$, and its two selecting bits are bit i of **lbits** and bit i of **rbits** — the same index into both arrays, read as aligned bit windows word-wise.

3.4 Use 1: round messages as gathers (the leaf round, worked)

A phase-A round accumulates three coefficients over slots b (with ω_b the driver's suffix-tensor eq weight — $V_j[b]$ in the code — and entries $2b, 2b+1$ per side):

$$A_0 = \sum_b \omega_b E[2b] O[2b], \quad A_2 = \sum_b \omega_b \Delta E_b \Delta O_b, \quad A_1 = \left(\sum_b \omega_b E[2b+1] O[2b+1] \right) + A_0 + A_2$$

(the A_1 form is the char-2 reading of $A_1 = H(1) - A_0 - A_2$). At the leaf layer the entries are *defined* as $E[i] = 1 + l_i \tau_L[i]$, $O[i] = 1 + r_i \tau_R[i]$ (l, r the committed bit halves). Expanding in char 2:

$$\begin{aligned}
\omega_b E[2b] O[2b] &= \omega_b + (l \cdot \omega_b \tau_L + r \cdot \omega_b \tau_R + lr \cdot \omega_b \tau_L \tau_R) && \text{a 4-case select,} \\
\Delta E_b &= E[2b+1] - E[2b] = l_{2b} \tau_L[2b] + l_{2b+1} \tau_L[2b+1] && \text{the 1s cancel,} \\
\omega_b \Delta E_b \Delta O_b &= \sum_{\varepsilon, \delta \in \{0,1\}} l_\varepsilon r_\delta \cdot \omega_b \tau_{L\varepsilon} \tau_{R\delta} && \text{a 16-case subset sum.}
\end{aligned}$$

The three tables. Every slot contribution is a subset sum of *eight* products — $\omega_b \tau_{L\varepsilon}$, $\omega_b \tau_{R\delta}$, $\omega_b \tau_{L\varepsilon} \tau_{R\delta}$ ($\varepsilon, \delta \in \{0,1\}$, all at slot b 's two positions) — plus a bare ω_b from each of the even and odd products. **build_leaf_tables** precombines the subset sums, once per τ set, at 8 multiplies (4 singles, 4 crosses) and ~ 20 adds per slot:

- **t_a0**[($b \ll 2$) | ($m_{L0} \mid m_{R0} \ll 1$)] — the 4 cases $\{0, \omega_b \tau_{L0}, \omega_b \tau_{R0}, \omega_b \tau_{L0} + \omega_b \tau_{R0} + \omega_b \tau_{L0} \tau_{R0}\}$ of the even product's non-constant part;
- **t_a1** — the same at the odd positions;
- the $\Delta\Delta$ table, **t_a2**[($b \ll 4$) | $c_l \mid (c_r \ll 2)$] — the 16 subset sums of the four cross products, selected by $m_{L\varepsilon} \wedge m_{R\delta}$;

and the constants collapse into one per-set scalar **w_sum** = $\sum_b \omega_b$ — tree-independent, added once at the end of the round. Together: $4 + 4 + 16 = 24$ entries (384 B) per slot, ≈ 12.6 MB at the $n = 28$ leaf layer ($k = 16$).

The slot body. A *tree*’s round is then, per slot:

$$\begin{aligned} c_l &\leftarrow \text{2-bit window of } \mathbf{lbits} \text{ at } 2b; & c_r &\leftarrow \text{same in } \mathbf{rbits} & (32 \text{ slots per } \mathbf{u64} \text{ read}) \\ A_0 &+= \mathbf{t_a0}[(b \ll 2) \mid c_{l,0} \mid c_{r,0} \ll 1], & S_1 &+= \mathbf{t_a1}[(b \ll 2) \mid c_{l,1} \mid c_{r,1} \ll 1], \\ A_2 &+= \Delta\Delta[(b \ll 4) \mid c_l \mid c_r \ll 2] \end{aligned}$$

— here S_1 collects the *odd product sum* $\sum_b \omega_b E[2b+1]O[2b+1] = H(1)$ (constants via $\mathbf{w_sum}$), and the round ends with the finalisation $A_1 = S_1 + A_0 + A_2$. Per slot this is three unconditional indexed loads and three XOR adds, **zero multiplications, zero branches** on committed bits (a branchy “skip the multiply by 1” variant was measured slower: 50/50 random bits make it mispredict-bound). The build is paid once per set; the gathers are paid per tree: $O(2^k)$ shared multiplies replace $O(2^k)$ multiplies *per tree*, an amortisation of *live* $\approx 2^8$.

Assembling the layer message. The triple a tree accumulates is its quadratic cofactor $H_c = (A_0, A_1, A_2)$. The driver multiplies each by the group’s running prefix scalar (seeded with $\mathbf{eq}(c, z_c)$, §2.2) and XORs across trees into *wide* unreduced 256-bit accumulators, with one polynomial reduction per accumulator per round — exact, because the reduction is $\mathbb{F}_2$ -linear. Since the $\mathbf{eq}$ factor of the round polynomial is public, the message sent is just the two non-constant coefficients of the summed cofactor (Gruen’s factored round format).

Two forms of the $\Delta\Delta$ table. The 16 $\Delta\Delta$ cases are subset sums of only four products, so the table ships in two interchangeable forms (**LeafA2**, auto-picked by footprint; **F2Z_LEAF_A2_FACTORED** forces): *precombined* — 16 entries per slot, one load and one add — or *factored* — only the four raw cross products $[p_{00}, p_{10}, p_{01}, p_{11}]$ per slot (one sequential 64-B line), consumed by four branchless masked adds that reduce pairwise, so the A_2 accumulator still takes one add. Factored shrinks the leaf tables from 24 to 12 entries per slot and wins outright once the precombined set outgrows the P-cluster L2 (the auto threshold, $2^{k-1} \geq 2^{15}$): at $n = 30$ the leaf round drops $2.1 \times (307 \rightarrow 147 \text{ ms})$; at the still-L2-resident $n = 26$ it loses $\approx 1.2 \text{ ms}$. Both forms accumulate identical field elements.

One level up: the 16-case pair body. At the pair layer the entries are themselves 4-case $\mathbf{te/to}$ selects, so a slot is keyed by four 2-bit cases $(c_{E0}, c_{E1}, c_{O0}, c_{O1})$, read as 2-bit windows at $2b$ and $2b + 2^k$ (**pair2_cases**; the E/O offset is even, so no window straddles a word). The $\Delta E \cdot \Delta O$ term now couples 8 bits — 256 cases, past the cap of §3.2 — so it is never fully precombined. Two exact forms exist (**Pair2Tables**):

- *precombined* (opt-out, **F2Z_PAIR2_FACTORED=0**): four 16-case tables per slot — $\mathbf{t_a0}[b \ll 4 \mid (c_{E0} \ll 2 \mid c_{O0})] = \omega_b TE_{2b}[c_{E0}] TO_{2b}[c_{O0}]$, the odd $\mathbf{t_a1}$, the weighted difference $\mathbf{t_wde} = \omega_b(TE_{2b}[c_{E0}] + TE_{2b+1}[c_{E1}])$, and the raw $\mathbf{t_do}$ — so the cross term is ONE wide multiply per slot; but the tables hold 64 entries per slot (16 MiB at the $n = 28$ pair layer, past L2), and the four gather streams’ misses dominate the round;
- *factored* (the default): only the suffix-weighted $\mathbf{wte}[i] = \omega_{i \gg 3} \cdot \mathbf{te}[i]$ is built (the layout of $\mathbf{te}$ itself, 8 entries per slot), and the body recombines against the set’s raw $\mathbf{to}$: per slot, the weighted even and odd wide products plus $(\mathbf{wte}_0 + \mathbf{wte}_1)(\mathbf{to}_0 + \mathbf{to}_1)$ — three wide multiplies against a $4 \times$ smaller, L2-resident table set.

The two agree exactly: $\omega TE_0 + \omega TE_1$ is the precombined $\omega(TE_0 + TE_1)$ (distributivity), the products keep the build’s association $(\omega TE) \cdot TO$, and the deferred reduction is $\mathbb{F}_2$ -linear. The

measured lesson generalises across this section: *multiplies are cheap next to bytes that miss*. This one body (`pair2_factored_body`) serves *four* rounds: round 1 of `Pair2Bits/Pair3Bits` over (`te`, `to`), and round 2 of `Leaf2Bits/Leaf3Bits` over the stashed value sets of §3.5 — only the case extraction differs.

3.5 Use 2: folds kept as tables (the stash)

Round 1’s fold should materialise round 2’s buffers — $\text{live} \cdot 2^{k-1}$ $\mathbb{K}$ -elements across the trees. It does not, because of one fact: *the fold of a bit-selected buffer is again a bit-selected buffer*. So the prover never writes the folded values — it writes down the new (shared) selection table and keeps using the bits.

Two representations of a buffer. A sumcheck operand can exist *dense* — an array of $\mathbb{K}$ -elements per tree, as §3.1 assumed — or in *case form*: one shared position-indexed table T plus the per-tree bits, under the convention that entry p of tree c ’s buffer *is defined as* $T[p]$ [a few of c ’s bits at fixed positions]. In case form the buffer is a *rule*, not memory: no per-tree $\mathbb{K}$ -element exists anywhere. At the leaf layer the buffers start in case form for free — $E[i] = 1 + m_i \tau_L[i]$ is the 2-case rule $\{1, 1 + \tau_L[i]\}$ selected by one bit — and §3.4 already showed how a *message pass* runs on that form, by gathers. What remains is the fold, and this is where the two representations differ: folding a dense buffer is a write pass *per tree*; folding a rule is a rewrite of the rule — one new table, no tree touched.

Folding the rule. Substitute the case form into the fold (6) at challenge ρ_1 (char 2 cancels the 1s, as in (7)):

$$E'[p] = (1 + \rho_1) E[2p] + \rho_1 E[2p+1] = 1 + m_{2p} (1 + \rho_1) \tau_L[2p] + m_{2p+1} \rho_1 \tau_L[2p+1].$$

The right-hand side is a function of the bit *pair* (m_{2p}, m_{2p+1}) , and its four possible values depend only on the position and ρ_1 — not on the tree. So the fold of *every tree at once* is the one shared table

$$F_1[p][m_0 | m_1 \ll 1] = 1 + m_0 (1 + \rho_1) \tau_L[2p] + m_1 \rho_1 \tau_L[2p+1]$$

(4 entries per position, `build_leaf_fold_tables`; one per side, F_1^L from τ_L , F_1^R from τ_R), and *executing* it per tree is optional. `LeafBits` does execute it — one F_1 gather per entry, writing dense round-2 buffers — but `Leaf2Bits` and `Leaf3Bits` instead *stash* F_1 : round 2’s buffer of tree c is henceforth *defined* as $p \mapsto F_1[p][m_{2p}, m_{2p+1}]$, and nothing per-tree is written. Round 2’s message pass runs on the rule by gathers: slot b needs entries $2b, 2b+1$, i.e. the aligned nibble $m_{4b..4b+3}$ of the *original* bits, split into two 2-bit cases (`leaf2_cases`). And by a layout coincidence the code exploits, “4 cases per position, two sides” is exactly the pair layer’s native value format (`Pair2TauSet`, `te` = F_1^L , `to` = F_1^R) — so round 2 *is* the pair-layer 16-case body of §3.4, run over the stash.

Iterating, and stopping. At ρ_2 the same substitution folds the 4-case rule into a 16-case one:

$$E''[q] = (1 + \rho_2) F_1[2q][c_0] + \rho_2 F_1[2q+1][c_1] =: F_2[q][(c_0 \ll 2) | c_1],$$

a function of the four original bits $m_{4q..4q+3}$ (`build_pair2_fold_tables`, 16 entries per position). `Leaf3Bits` stashes F_2 as round 3’s value tables; round 3 gathers with byte windows ($m_{8b..8b+7}$ per slot, the nibble halves swapping into the index via `leaf3_idx`) and runs the plain dense body over

the gathered values — at 8 selecting bits a precombined slot table would cost more bytes than the multiplies it saves (§3.2). There the chain stops: ρ_3 ’s fold would need a 256-case F_3 — each stash step *squares* the cases while only halving the positions, so the shared table *doubles* every step, the width law of §3.2 marching toward its cap — so round 3’s fold is *executed*: per tree, gather the two F_2 values of each slot and write $(1+\rho_3)(\cdot) + \rho_3(\cdot)$ into real arrays. These are the first per-tree $\mathbb{K}$ -elements of the whole layer, 2^{k-3} per side; rounds 4 onwards are ordinary dense sumcheck. **Pair2Bits/Pair3Bits** are the same chain started one level up, where the initial buffer is already a 2-bit-selected rule (**te,to**), so the counting begins at 4 cases and the cap arrives one round sooner (**pair3_idx** keys the stashed round-2 rule).

The prover’s physical state, round by round. For **Leaf3Bits**, one τ set; table sizes count both sides, at the $n = 28$ leaf layer ($k = 16$, *live* = 2048):

	round j ’s buffer exists as	shared table	at $k=16$
round 1	bits + τ halves: $1 + m_i \tau[i]$	$2 \cdot 2^k$ entries	2 MiB
round 2	bits + $F_1[p][m_{2p}, m_{2p+1}]$	$2 \cdot 4 \cdot 2^{k-1}$	4 MiB
round 3	bits + $F_2[q][m_{4q}, m_{4q+3}]$	$2 \cdot 16 \cdot 2^{k-2}$	8 MiB
rounds 4+	dense per-tree buffers	—	512 MiB

Until round 3’s fold, the only per-tree state is the original bit array; the challenges live inside the tables. The cascade, per driver entry (“stash” = the round’s fold is kept as the next round’s value tables; “dense (·)” = the fold materialises buffers of that size per side per tree):

entry	needs	round 1	round 2	round 3
LeafBits	$k \geq 2$	leaf LUT	dense (2^{k-1})	
Leaf2Bits	$k \geq 3$	leaf LUT, stash	pair body	dense (2^{k-2})
Leaf3Bits	$k \geq 4$	leaf LUT, stash	pair body, stash	inline; dense (2^{k-3})
Pair2Bits	$k \geq 2$	pair body	dense (2^{k-1})	
Pair3Bits	$k \geq 3$	pair body, stash	inline; dense (2^{k-2})	
T4Bits	$k \geq 2$	inline $T4$; dense (2^{k-1})		

The “needs” column is §3.2’s structural guard ($k \geq f+1$: the last LUT round must still have a fold, where the dense buffers get born). **F2Z_LUT3=0** forces the shallower **Pair2/Leaf2** prefixes at every depth — the same path depth-4 trees take; depth-3 trees run **LeafBits** alone.

Reading the cases. Each round down a cascade reads a wider aligned bit window per slot, always keyed by the *original* committed bits — nothing per-tree is ever written between rounds, which is the point:

round body	windows per slot	helper
leaf round 1	two 2-bit fields at $2b$	inline
Leaf2/Leaf3 round 2	two nibbles at $4b$	leaf2_cases
Leaf3 round 3	two bytes at $8b$	leaf3_idx
Pair2/Pair3 round 1	four 2-bit fields at $2b, 2b+2^k$	pair2_cases
Pair3 round 2	four nibbles at $4b, 4b+2^k$	pair3_idx
T4Bits round 1	16 single bits	t4bits_idx

The windows are §3.1’s contiguous-block fact made physical: after f folds an entry’s selecting bits are one aligned $2^f \cdot w$ -bit block, and the E/O offset 2^k stays even at every round, so no window ever straddles a word.

Economics. The count that decides everything: an executed fold stores one value per position *per tree*; a stashed one stores 4 (resp. 16) values per position *in total*. Materialising at round j thus costs $live \cdot 2^{k-j}$ values per side; the stash costs $4 \cdot 2^{k-j}$ resp. $16 \cdot 2^{k-j}$ entries per side *per τ set* — factors $live/4$ and $live/16$, i.e. $512\times$ and $128\times$ at the deployed $s = 11$. Concretely, at the $n = 28$ leaf layer ($k = 16$, $live = 2048$): executing round 1’s fold would write 2 GiB of round-2 buffers, where the stash is 4 MiB; round 2’s stash is 8 MiB against a 1 GiB materialisation; and the first dense buffers, at round 3’s fold, are $2^{n-3} \cdot 16 \text{ B} = 512 \text{ MiB}$. It is this LUT prefix depth, not the storage schedule, that fixes the bottom layers’ resident set.

3.6 Use 3: generating the dense region from $T4$

Above the pair layer the entropy argument runs out ($r + f \leq 2$), so $T4$ is used not to *avoid* buffers but to *create* them cheaply — and the remaining freedom, which level is stored, which regenerated, which implicit, is the memory schedule.

The stored chain, and the build. `build_levels` builds each tree’s upper levels bottom-up by repeated `parent_halves_top` ($v[i] = E[i] \cdot O[i]$, re-split at the new midpoint), parallel across trees, then regroups the chains level-major; the roots fall out of level 1. The chain’s *top* level is produced directly from bits and $T4$ (`gen_top`), one subcube of the level-width observation (§3.2) per position: under $L/2$ one $T4$ gather (= level $d-2$); under $L/4$ the product of the pair $\{y, y+2^{d-3}\}$ — 2 gathers, 1 multiply (= level $d-3$); under $L/8$ the quadruple $\{y, y+2^{d-4}, y+2^{d-3}, y+3 \cdot 2^{d-4}\}$ — 4 gathers, 3 multiplies (= level $d-4$). Levels d and $d-1$ (and, below $L/2$, $d-2$) therefore never exist at build time: each of their positions is gathered exactly once, with the materialise-then-read round trip removed. The chain that remains sums to $\approx 2^{\text{top}+1}$ values per tree — a quarter of the leaf count under $L/4$, an eighth under $L/8$, whence the names — and `drive_grouped` takes each level with `mem::take` as its layer starts, so the chain shrinks as the forest climbs. The stored chain and the JIT level below it are equal-sized and never simultaneously live.

The JIT level, and fused generation. Under $L/4$, layer $d-3$ regenerates its input level $d-2$ per tree (2^{d-3} values per half, one 4-bit-indexed $T4$ gather each) into dense buffers that die when the layer ends; under $L/8$, layer $d-4$ does the same for level $d-3$ at two gathers and a multiply per value. By default the generation pass is not just a write: `jit_layer_generate` accumulates the layer’s opening round(s) *as it writes*. In the plain form (`dense_jit_fused_round1`) that is round 1’s coefficient triple; with the driver’s double-fold on and $k \geq 3$ (`dense_jit_fused_grid`) it is the whole 3×3 bivariate grid $G(X_1, X_2) = \sum_b V(b) L(X_1, X_2, b) R(X_1, X_2, b)$ — nine field elements per tree, from which round 1’s message $\sum_{x_2} \text{eq}_1(x_2; q) G(X_1, x_2)$ and round 2’s $G(\rho_1, X_2)$ both derive, so the level’s first actual read is round 3’s pass, which then folds two deferred challenges at once. Either way the result reaches the driver through its `pre_round1` hook and is value-exact against the passes it replaces: XOR accumulation is order-free and the deferred reduction $\mathbb{F}_2$ -linear.

The 4-leaf round ($L/8$ only). One level further up, `T4Bits` runs a whole layer off bits: each entry is a single inline 16-case $T4$ gather — `t4bits_idx` assembles $(j \ll 4) \mid (c_E \ll 2) \mid c_O$ from four committed bits read straight from the transposed leaf halves at j and $j + 2^{k+1}$ — with dense buffers at its fold. The layer’s input level is neither stored nor regenerated; combined with the JIT level below it, this is what makes $L/8$ an eighth.

The three schedules. The schedules differ in nothing but the fate of levels $d-2$ and $d-3$:

	L/2 (12)	L/4 (default)	L/8 (18)
chain tops at	level $d-2$	level $d-3$	level $d-4$
per top position	1 $T4$ gather	2 gathers, 1 mul	4 gathers, 3 muls
level $d-2$	stored	JIT at layer $d-3$	implicit (<code>T4Bits</code>)
level $d-3$	stored	stored	JIT at layer $d-4$
peak resident	$\approx 2^n \cdot 8\text{ B}$	$\approx 2^n \cdot 4\text{ B}$	$\approx 2^n \cdot 2\text{ B}$
needs depth	≥ 4	≥ 4	≥ 5 (else L/4)

Schedule choice is a near-pure *memory* decision. The measured rule of thumb: one $T4$ gather sweep costs about one streaming pass over the level it would have saved — so L/2, which removes the JIT sweep but doubles the resident set, comes out flat to worse than L/4, while L/8 costs +5–13% of prove and is what makes $n \geq 31$ fit a 16 GB box. The time levers live inside the round bodies of §§3.4–3.5, not in the ladder.

Only live columns are built. A committed column whose bits are all zero has every leaf $\alpha^0 = 1$, hence every level and its root = 1. Since a phase-A round message is $\sum_c \text{eq}(c, z_c) H_c$ and every constant-1 tree has the *same* H_c , the trailing run of zero columns collapses into *one* synthetic group with all-ones buffers and scale $\sum_{c \geq \text{live}} \text{eq}(c, z_c)$ (`col_elide`, `live_cols`). The absorbed root vector is still padded with the known 1s, so the statement is unchanged; on a JIT layer the synthetic group is born inside the generation pass. Char-2 re-association is exact, so the transcript is byte-identical to the un-elided forest (`col_elision_matches_full`): a witness of N cells padded to 2^n pays the forest for $\lceil N/2^{t+\log_2 W} \rceil$ columns instead of 2^s . Everything in this section runs over the live leading trees only.

The ladder, per layer. Seen from the layers, the default schedule reads:

layer	input level	entries exist as	LUT rounds	dense born
$d-1$ (leaf)	leaves	$1 + m\tau$ over the τ halves	1–3 (2-bit, nibble, byte)	round 3’s fold
$d-2$ (pair)	$d-1$	4-case <code>te/to</code> selects	1, 2	round 2’s fold
$d-3$	$d-2$	JIT from $T4$ gathers	(fused into generation)	at generation
$\leq d-4$	$\leq d-3$	stored chain	—	prebuilt

3.7 Why the transcript is unchanged

Every device above is a re-association of the same field arithmetic: precombined entries are subset sums the dense path computes per entry (distributivity); the char-2 identities $1 + 1 = 0$ and $(1+\rho) + \rho = 1$ are what cancel the leaves’ 1s and keep folded entries affine; XOR accumulation is order-free and the deferred wide reduction $\mathbb{F}_2$ -linear. Same field elements, same transcript order; the verifier and the proof codec never learn which path ran. The byte-identity is pinned in every schedule and depth regime (`lazy_matches_eager`, `lazy_multi_matches_eager`, `col_elision_matches_full`).

4 Where this sits in the code

this note	code
the layer loop (§2.2)	<code>drive_grouped / verify_merged_forest</code>
per-layer proof (phase A, phase B, (p, q))	<code>MergedLayer (sc_x, sc_c, pair)</code>
the line challenge λ	<code>mu</code>
the integer folds u_c	<code>fold_values_bits</code> ; proof field <code>us</code>
α -power table $v(i)$	<code>chunk_pow2_table</code>
R table / m table (§2.3)	<code>row_bit_weights / xi_combined_rows</code>
the presum	<code>MultiDegreeSumcheck</code> in <code>prove/verify_int_eval_merged_common</code>
range/generator/root checks, μ	<code>verify_int_eval_merged_common</code>
extraction	
chunk loop, η -RLC, the one Ligerito call	<code>prove/verify_mle_eval_mod_q_ligerito</code>
the lazy prover	<code>prove_merged_forest_lazy</code>
LUT round bodies (§3.4–3.5)	<code>GroupBufs: LeafBits, Leaf2Bits, Leaf3Bits, Pair2Bits, Pair3Bits, T4Bits</code>
leaf tables <code>t_a0/t_a1/$\Delta\Delta$/w_sum</code>	<code>build_leaf_tables</code>
fold/stash tables	<code>build_leaf_fold_tables, build_pair2_fold_tables</code>
shared level tables	<code>leaf_tau_halves, layer1_pair_table, te/to, T4</code>
JIT generation (fused round 1 / grid)	<code>jit_layer_generate</code>

For the checks in full detail see `docs/verifier-note/`; for the schedules, kernels, fusion, and the measured cost tables see `docs/forest-gkr-note/`; for the surrounding pipeline (commit, ring switch, Ligerito, serialization) see `docs/DESIGN.md`. The standard caveats carry over: flock’s Merkle domain separation is a flagged upstream item, the ad-hoc Ligerito config regime is unaudited, and the scheme is not zero-knowledge.

References

- [1] S. Goldwasser, Y. T. Kalai, G. N. Rothblum. *Delegating computation: interactive proofs for muggles*. J. ACM 62(4), 2015 (STOC 2008).
- [2] J. Thaler. *Time-optimal interactive proofs for circuit evaluation*. CRYPTO 2013. (The product-tree GKR specialisation the layers instantiate.)
- [3] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, D. Song. *Libra: succinct zero-knowledge proofs with optimal prover computation*. CRYPTO 2019. (The eq-factored linear-time prover lineage.)
- [4] L. Soukhanov. *The characteristic-2 field-switching note*, §9. (The integer-MLE-over- $\mathbb{F}_2$ construction F2Z realises; internal.)